ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А. Э. Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Интеграция общего (системного) и специального программного обеспечения в программных комплексах.

по курсу: Основы программной инженерии

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ

гр. № 4434		
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ЗАДАНИЕ 4

ВВЕДЕНИЕ 5

1 Лабораторная работа №3 — реализация и интеграция 6

1.1 Запуск 6

1.2 Структура и поведение 7

1.3 Локальная разработка без Docker 9

1.4 Демонстрация 10

1.5 Проверки и эксплуатация 10

1.6 Роли и выводы 11

2 Контракт API 12

3 Проектирование и эксплуатация базы данных 14

3.1 Состав таблиц и связи 14

3.2 Роли и доступ 14

3.3 Индексы для порядка 100 тысяч объектов 15

3.3.1 Оценка ёмкости 16

3.4 Транзакции и ограничения 17

3.5 Резервное копирование и восстановление 17

4 Проверка ЛР №3 19

Серверные интеграционные тесты 19

Проверки клиента 19

Нагрузочный набор и измерения 20

Системная интеграция 20

Автоматический отдельный тестовый стенд 21

ЗАКЛЮЧЕНИЕ 22

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 23

ПРИЛОЖЕНИЕ А Результаты проверки интерфейса..... 24

ЗАДАНИЕ

Разработать Web-интерфейс, серверную обработку и базу данных по требованиям первой работы и задачам второй.

Задание выдал: _____

Подпись: _____

Задание принял: _____

Дата: _____

ВВЕДЕНИЕ

Работа реализует принятые требования супермаркета на React, FastAPI и PostgreSQL. Основные проверяемые свойства — серверная валидация, разграничение доступа, сохранение исторических цен, атомарная оплата и постраничная работа с большим каталогом.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

1 Лабораторная работа №3 — реализация и интеграция

Участники: A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим.

Вариант 36: информационная система супермаркета. Здесь находятся прикладные исходники и схема БД. Требования и архитектура приняты на этапе ЛР №1, среда и порядок совместной работы подготовлены на этапе ЛР №2.

1.1 Запуск

В Linux VM с Docker Compose v2 и openssl:

```
cd 3lab
bash setup.sh
docker compose --env-file .env ps
curl --fail http://127.0.0.1:8083/api/health
```

Интерфейс: http://127.0.0.1:8083. OpenAPI/Swagger: http://127.0.0.1:8083/api/docs. Пользователи: manager, cashier, auditor. Их пароли сгенерированы в локальном .env в полях MANAGER_PASSWORD, CASHIER_PASSWORD, AUDITOR_PASSWORD. Просматривать их локально, не включать в Git и скриншоты отчёта. Они не зашиты в приложение. Токен хранится только в памяти страницы; обновление страницы требует повторного входа.

Начальная загрузка создаёт 16 демонстрационных товаров, трёх пользователей и движения начального остатка. Это небольшой набор для знакомства с UI; отдельный сценарий 100 тысяч объектов находится в db/seed_large.sql. Повтор bootstrap не меняет существующие пароли, карточки или остатки. Товарные цены и данные вымышленные учебные.

Используются Python 3.12, FastAPI, Pydantic 2, psycopg 3, PostgreSQL 17, React 18.3, JavaScript/JSX, PropTypes, Vite и Nginx. React 18.3 выбран для автоматических проверок PropTypes в development; в React 19 они удалены. В production дополнительная проверка JSON в api.js остаётся обязательной.

1.2 Структура и поведение

init.sql создаёт роли, таблицы, ограничения, индексы и триггеры; db/10-init.sh передаёт пароли из окружения через psql. backend/app/main.py только собирает FastAPI-приложение и управляет пулом. routers/ содержит HTTP-контракты, services/ — операции каталога, остатков и чеков, dependencies.py — сеанс и роли, errors.py — единые ответы об ошибках, schemas.py — Pydantic-валидацию, security.py — пароли и токены. seed.py выполняет идемпотентную начальную загрузку.

1.2.1 Компонентная структура Web-клиента

Пользовательский интерфейс декомпозирован на независимые функциональные компоненты, представленные в таблице 1. Внешний вид ключевых экранов и диалоговых окон системы приведён в приложении А (рисунки А.1–А.8).

Таблица 1 — Функциональные компоненты клиентской части системы

Компонент / Файл	Уровень декомпозиции	Принимаемые входные параметры (Props)	Назначение и функционал
App.jsx	**Root Layout / State Coordinator**	**—**	**Координация глобального сеанса, хранение активного токена и профиля сотрудника, переключение табов, двухэтапный сценарий кассы (draft → pay)**
Login.jsx	**Page / Auth**	**onLogin: PropTypes.func.isRequired**	**Форма авторизации, отправка POST /api/auth/login, валидация логина по регулярному выражению, блокировка кнопки на время запроса**
WorkspaceNav.jsx	**UI / Sidebar**	**tab, user, onCatalog, onReceipts, onLogout**	**Боковая панель навигации, индикация текущей роли сотрудника, выход из системы с отзывом сеанса**
Catalog.jsx	**Feature / Page**	**token, onAdd, onEdit, onDetails, cart, refreshKey, disabled**	**Каталог товаров: поисковая строка с задержкой (debounce 300 мс), фильтрация по категориям, курсорная пагинация (limit=24)**
ProductCard (в Catalog.jsx)	**UI Component**	**product: productShape, onAdd, onEdit, onDetails, quantity, disabled**	**Карточка товара: фото упаковки, наименование, штрихкод, категория, складской остаток, цена, кнопки управления**
ProductDetailsDialog.jsx	**UI / Dialog**	**productId, token, onClose, onUnauthorized**	**Модальное окно детализации: адаптивное изображение упаковки, пищевая ценность (КБЖУ), векторный SVG-штрихкод EAN-13**
ProductEditor.jsx	**Feature / Dialog**	**product, token, onClose, onSaved**	**Экран менеджера: создание/редактирование карточки товара, версионирование (version), ручная корректировка остатка с фиксацией основания**
CartPanel.jsx	**Feature / Aside**	**cart, draft, busy, creating, locked, onChange, onPrepare, onFinish**	**Кассовый узел: корзина покупателя, фиксация черновика чека, выбор типа оплаты, расчёт сдачи при оплате наличными**
ReceiptHistory.jsx	**Feature / Page**	**userRole, receipts, cursor, busy, selected, saleAllowed, onLoadMore, onInspect**	**Журнал чеков: курсорная пагинация (DESC), разграничение доступа по ролям, просмотр реквизитов и состава архивного чека**

В клиентской части реализован ряд обязательных архитектурных решений:

1. Защита от потери синхронизации ввода (Race Condition): в компоненте Catalog.jsx введена задержка ввода (debounce 300 мс), а устаревшие сетевые запросы при быстром вводе текста отменяются посредством интерфейса AbortController.
2. Идемпотентность кассовых операций: перед отправкой чека на сервер генерируется UUID-ключ (crypto.randomUUID()), исключающий повторное списание средств при сбоях канала связи.
3. Векторная отрисовка штрихкодов: компонент детализации товара выполняет математический расчёт контрольной суммы кода EAN-13 по алгоритму Modulo 10 и динамически формирует разметку SVG без использования внешних библиотек.
4. Доступность и управление фокусом: модальные окна закрываются по нажатию клавиши Escape, реализована навигация клавишей Tab, кнопки выполнения асинхронных действий блокируются (disabled) до получения ответа сервера.

У роли приложения нет права менять структуру БД, удалять товарные документы или изменять пользователей. Приложение разграничивает действия пользователей отдельно от прав SQL. В конфигурации pg_hba разрешены только нужные пользователи для нужной БД; внешняя сеть закрыта. Локальный trust для postgres внутри контейнера нужен официальному entrypoint при bootstrap, не даёт удалённого входа.

Продажа двухэтапная: «Сформировать чек» фиксирует состав и цены, «Наличные получены» проводит оплату и списание в общей транзакции. Отмена применяется только к черновику. Реальный эквайринг и фискализация не реализуются. Историческая цена не меняется вслед за каталогом.

1.3 Локальная разработка без Docker

Создать отдельную PostgreSQL-БД supermarket, запустить init.sql администратором кластера. Пароли передавать переменными psql

app_password, admin_password; файл не содержит готовых паролей. Для автоматизации предпочтителен включённый контейнерный bootstrap. Затем:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export DB_HOST=127.0.0.1 DB_PORT=5432 DB_NAME=supermarket DB_USER=market_app
read -rs -p 'Пароль market_app: ' DB_PASSWORD; export DB_PASSWORD
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Первичное заполнение python -m app.seed выполнять отдельно с DB_USER=market_admin, его паролем и тремя переменными паролей сотрудников. После bootstrap API запускается только под market_app. Для frontend — Node 22.12+:

```
cd frontend
corepack enable
pnpm install --frozen-lockfile
pnpm dev
```

Vite проксирует /api в localhost:8000, поэтому не нужен CORS *. Production: pnpm build. Lockfile обязателен. Приложение не зависит от файлов 1lab и не использует заглушку 2lab.

1.4 Демонстрация

1. Войти менеджером (рисунок А.1); создать карточку с уникальным кодом; добавить остаток с основанием.
2. Войти кассиром (рисунок А.1); найти товар по названию/штрихкоду; добавить в корзину; сформировать чек.
3. Проверить серверную сумму и подтвердить наличные; в истории открыть оплаченный чек.
4. Повторить запрос оплаты: остаток не должен уменьшиться второй раз.
5. Создать два черновика на последний экземпляр; одновременная оплата завершает только один.
6. Продемонстрировать 422 на отрицательную цену/дробное количество, 403 на изменение товара кассиром, запрет отмены оплаченного чека.

1.5 Проверки и эксплуатация

Проверка работоспособности и эксплуатационных характеристик системы проводилась по многоуровневому регламенту, включающему

серверные интеграционные тесты, проверку доступности клиентского интерфейса и нагрузочные измерения базы данных. Персистентность данных при перезапуске контейнеров обеспечивается внешним томом СУБД. При модернизации структуры данных регламент предусматривает предварительное создание резервной копии и последовательное применение миграций. Скрипт инициализации схемы ориентирован на первичное развертывание чистой базы данных.

Пакет является учебной версией. Для внешней эксплуатации нужны TLS, отдельный процесс управления сотрудниками, регламент мониторинга, резервного копирования и ротации секретов. Приложение слушает localhost по умолчанию. Swagger подгружает UI-ресурсы с CDN; сама схема /api/openapi.json доступна без CDN.

1.6 Роли и выводы

A — администратор БД: таблицы, роли, pg_hba, индексы и резервное копирование.

B — разработчик Web и мультимедийных приложений: FastAPI, GET/POST, серверная валидация, интеграция с PostgreSQL.

C — специалист по дизайну графических пользовательских интерфейсов: HTML/CSS, JSX, хуки, PropTypes, состояния интерфейса.

Согласованное изменение стека заменяет CGI/C++ и примеры Apache связкой FastAPI/Python и Nginx, но сохраняет требования методички к серверной проверке, GET/POST, чтению/записи БД и различным ролям. Основание: методичка ЛР №3, с. 7, 19–25, 57–63. Вывод: Web-интерфейс интегрирован с серверной логикой и БД, а целостность продажи обеспечивается сервером и транзакциями.

2 Контракт API

Базовый путь /api. Защищённые методы требуют Authorization: Bearer TOKEN. Токен выдаётся POST /auth/login, сохраняется на сервере в виде SHA-256 и действует 8 часов. POST /auth/logout отзывает текущую сессию; GET /auth/me возвращает идентификатор, имя и роль. Пароль проверяется scrypt с индивидуальной солью. Лимит попыток входа настроен в Nginx, API не опубликован напрямую.

Метод и путь	Роль	Результат
GET /health	Без входа	Проверка соединения с БД
POST /auth/login	Без входа	access_token, token_type, expires_at
GET /auth/me	Любая	Текущий сотрудник
POST /auth/logout	Любая	204, токен отозван
GET /products	Любая	items и next_cursor
POST /products	manager	201, созданная карточка с stock=0
PUT /products/{id}	manager	Полная карточка с новым version
POST /products/{id}/stock	manager	Новый остаток и версия
POST /receipts	cashier, manager	201, черновик или прежний результат ключа
GET /receipts	Любая	Страница истории; для кассира только собственные
GET /receipts/{id}	По правам чтения	Чек, строки, итог
POST /receipts/{id}/pay	cashier, manager	Оплаченный чек; повтор безопасен
POST /receipts/{id}/cancel	cashier, manager	Отменённый черновик

Каталог: q — подстрока названия от 3 символов либо полный цифровой штрихкод длиной 8–14; category — точное название категории; after — предыдущий ID, по умолчанию 0; limit — 1–100, по умолчанию 24; include_archived=true доступен менеджеру. Порядок всегда по ID ASC. Для истории — before и ID DESC. next_cursor=null означает конец. Пагинация не представляет снимок БД между запросами: параллельные вставки или изменения могут изменить видимый набор, но offset-смещения нет.

Пример создания карточки:

```
{"barcode":"4601234567890","name":"Молоко 1 л","category":"Молочные продукты","price":"99.90"}
```

При PUT передаются те же поля, active и текущий version. Начальный остаток 0; корректировка выполняется отдельно:

```
{"delta":10,"reason":"Учебное поступление"}
```

Создание чека (ID товара взять из реального GET/POST):

```
{"request_id":"78c9bf7f-06e5-451a-a81a-e528537296e8","items":[{"product_id":1,"quantity":2}]}
```

Цена, итог, статус, автор, ID чека и paid_at не принимаются от клиента. Неизвестные поля запрещены. Повтор с тем же request_id и тем же составом возвращает тот же документ, в том числе если он уже завершён. При неизвестном исходе запроса нельзя автоматически генерировать другой UUID.

Сервер возвращает цены/суммы строками Decimal ("199.80"), даты — ISO 8601 с часовым поясом. Корзина браузера имеет только предварительную сумму. Гарантия исторической цены начинается после создания серверного чека.

Ошибки валидации имеют detail: [{loc, msg, type}] без исходного тела и пароля. Бизнес-ошибки имеют detail: "сообщение". 409 не повторяют бесконечно: нужно обновить данные или отменить черновик. 503 при блокировке допускает повтор того же идентификатора. 401 означает необходимость повторного входа.

3 Проектирование и эксплуатация базы данных

3.1 Состав таблиц и связи

products — товары; receipts — чеки; receipt_items — строки; stock_movements — движения; app_users — сотрудники; sessions — сеансы. Имена SQL латинские, предметные русские наименования приведены в комментариях и документации. Две таблицы «Товары» и «Чеки» не могут корректно представить многопозиционную покупку без связующей таблицы; поэтому добавлена receipt_items с PK(receipt_id, product_id).

У сотрудника много чеков и сеансов. В чеке 1–100 строк. Товар встречается во многих чеках; одна пара товар–чек уникальна. Движение относится к товару и сотруднику, а при продаже — к чеку. UNIQUE(receipt_id, product_id) не допускает два движения одного товара одной продажи; у ручных движений receipt_id NULL, поэтому они могут повторяться.

Деньги хранятся numeric(8,2), стоимость строки numeric(14,2) вычисляется БД; итог суммируется из строк при чтении. Копирование цены и наименования в строку сохраняет исторический документ при изменении товара. Остаток в карточке и журнал движений изменяются в одной транзакции. DB CHECK не заменяет серверную проверку точности входной цены: PostgreSQL numeric может округлить значение при прямой записи, поэтому до БД Pydantic запрещает лишние десятичные знаки.

3.2 Роли и доступ

Роль	Тип	Права
market_owner	NOLOGIN	Владелец схемы, таблиц и функций
market_admin	LOGIN, член market_owner	Управление схемой и начальными данными; не суперпользователь кластера
market_app	LOGIN	Чтение нужных таблиц, вставка карточек/чеков/движений/сеансов, обновление разрешённых столбцов, удаление сеансов

Роль	Тип	Права
market_reader	NOLOGIN	Групповые права чтения товаров, чеков, строк и движений без хешей паролей

Администратор может создать персональный LOGIN и включить его в market_reader; пароль задаётся через \password, а не хранится в SQL. PUBLIC лишается CONNECT к БД и CREATE в public. Предоставление новых таблиц не делает их автоматически доступными market_app: миграция должна явно выдать права. Приложение не работает под postgres или владельцем схемы.

Прикладные роли manager/cashier/auditor — значения app_users.role, а не отдельные PostgreSQL-соединения. Разграничение API проверяется на сервере; технические права market_app дополнительно ограничивают ущерб от ошибки. Прямой административный SQL остаётся доверенной операцией.

3.3 Индексы для объектов

Индекс	Запрос	Обоснование
PK products(id)	Карточка и соединения	Точечный доступ и стабильный ключ
UNIQUE products(barcode)	Полный штрихкод	B-tree, уникальность и быстрый equality
products_active_id_idx	Активный каталог id > cursor ORDER BY id	Частичный индекс исключает архив
products_category_id_idx	category=... AND active AND id>cursor	Фильтр и порядок в одном индексе
products_name_trgm_idx	lower(name) LIKE '%строка%' AND active	GIN pg_trgm для поиска внутри названия
receipts_cashier_id_idx	Свои чеки по ID DESC	Составной B-tree без сканирования всей истории
receipts_created_idx	История по времени для отчётного SQL	Период и устойчивый порядок
PK receipt_items(receipt_id,product_id)	Состав чека	Один диапазон индексного поиска
receipt_items_product_idx	Продажи товара	Индекс внешнего ключа и аналитика товара
stock_movements_product_time_idx	История движения товара	Поиск по товару и времени

Индекс	Запрос	Обоснование
sessions_expiry_idx	Очистка истёкших сессий	Ограничивает поиск удаляемых записей

Поиск от трёх символов снижает стоимость малоизбирательных шаблонов. Для редкой подстроки планировщик может выбрать GIN/Bitmap, для широкого запроса с небольшим LIMIT — B-tree по ID, а для большого покрытия — последовательное чтение. Наличие индекса не обязывает планировщик использовать его.

Коллация market.unicode_case создаётся через ICU с locale und. Запрос и индекс используют одинаковое выражение lower(name COLLATE unicode_case). Это обеспечивает поиск русских названий без учёта регистра даже при системной локали C. PostgreSQL должен быть собран с ICU, как выбранный официальный образ; смешанный регистр проверяется интеграционным тестом.

Пагинация id > cursor, максимум 100 записей; полный COUNT(*) не выполняется на каждую страницу. В памяти браузера хранится текущая страница. OFFSET для глубокого обхода не применяется. При 100 тысячах товаров партиционирование не обязательно: сначала измеряются EXPLAIN (ANALYZE, BUFFERS), размер таблиц, частота записи и latency API.

3.3.1 Оценка ёмкости

При условных 200–400 байтах на строку товара 100 тысяч товаров дают 20–40 МБ полезных строк без точного учёта TOAST и свободного места; индексы увеличивают объём. 100 тысяч чеков × 5 строк — 500 тысяч строк продаж и столько же движений. При 150–250 байтах строки это десятки/сотни мегабайт плюс индексы и WAL. Это инженерная оценка порядка, не замер. Для VM резервируется 40 ГБ, поскольку ОС, образы, резервные копии и временные файлы могут занимать больше собственно БД.

seed_large.sql создаёт 100 тысяч дополнительных товаров и 100 тысяч черновиков по 5 строк, фиксирует фактическое количество и

pg_total_relation_size. Набор синтетический; его нельзя загружать в рабочую базу. Он проверяет объём и планы чтения, а конкурентные оплаты проверяются отдельным API-тестом. После генерации выполняется ANALYZE.

3.4 Транзакции и ограничения

Оплата начинает транзакцию, блокирует чек FOR UPDATE и проверяет состояние. При paid возвращает существующий результат. Затем блокирует товары в порядке ID, проверяет активность и остаток, создаёт отрицательные движения, уменьшает stock и завершает чек. Возникший 409 откатывает транзакцию. Это предотвращает потерянные обновления и перепродажу последнего экземпляра. Блокировки PostgreSQL.

Триггер запрещает изменение завершённого чека и оплату пустого. Триггер вставки строки блокирует родительский чек, разрешает только draft и максимум 100 строк. Права market_app запрещают изменение/удаление строк чека. Состояние paid эквивалентно наличию paid_at. Смена цены не меняет unit_price в старых строках.

3.5 Резервное копирование и восстановление

Команды из 3lab для запущенного Compose:

```
mkdir -p backups
docker compose exec -T db pg_dump -U postgres -d supermarket -Fc > backups/supermarket.dump
docker compose exec -T db pg_restore --list < backups/supermarket.dump
docker compose exec -T db createdb -U postgres supermarket_restore
docker compose exec -T db pg_restore -U postgres -d supermarket_restore --exit-on-error <
backups/supermarket.dump
docker compose exec -T db psql -U postgres -d supermarket_restore -c 'SELECT count(*) FROM
market.products;'
```

Это восстановление в новую БД того же кластера, где роли уже существуют. При восстановлении в другой кластер сначала восстановить/создать роли; pg_dump не включает глобальные роли. Для них отдельная защищённая копия pg_dumpall --roles-only, содержащая чувствительные сведения. Доступ к новой базе ограничить теми же CONNECT/pg_hba-правилами; восстановление не означает автоматического переключения API.

Ежедневная копия и периодическая проба восстановления обеспечивают проверяемость целевых RPO/RTO; без фактического запуска расписания и восстановления достижение этих целей не утверждается. Копии хранить вне тома основной БД; удаление старых копий — отдельная явно определённая политика.

4 Проверка ЛР №3

Серверные интеграционные тесты

Тесты используют настоящую PostgreSQL, а не SQLite. Нужна отдельная БД с именем, заканчивающимся _test, и схема init.sql. На сервере с несколькими тестовыми базами учитывать, что роли PostgreSQL относятся ко всему кластеру: создавать их один раз, а дальнейшие схемы восстанавливать из проверенной копии.

DATABASE_URL — строка подключения market_app, TEST_ADMIN_URL — market_admin в той же отдельной базе. Передать секреты через окружение. Из backend:

```
python -m unittest discover -s tests -v
```

Без переменных тесты явно пропускаются и не означают успешную интеграционную проверку. Fixture создаёт пользователей test_manager, test_cashier, test_auditor и меняет только их пароли в _test-БД. Товары имеют случайные штрихкоды; существующие таблицы не очищаются. После тестирования временный стенд можно удалить целиком отдельно от рабочих данных.

Проверяются: аутентификация/выход, роли, некорректные деньги и количества, чужие поля, пустой и дублированный состав, ограничение пагинации, SQL-подобная строка поиска, повтор UUID, повтор pay, снимок цены, конкуренция за последний товар, переходы состояний, архивирование, конфликт версии карточки и ограниченные SQL-права.

Проверки клиента

Из frontend: запуск тестов выполняется командой pnpm test, сборка - pnpm build. Набор из 9 автоматических тестов (фреймворк Vitest, React Testing Library в среде jsdom) проверяет: блокировку добавления при нулевом остатке (stock = 0), обработку и повтор сетевых сбоев, курсорную пагинацию без смещения, перехват некорректного JSON функцией validateProductPage и автоматический выход при коде 401. Дополнительно проверена доступность:

навигация клавишами Tab/Shift+Tab и закрытие диалоговых окон клавишей Escape.

Нагрузочный набор и измерения

В отдельной _test или _bench БД после начальной загрузки выполнить db/seed_large.sql, затем db/explain.sql. Записать actual time, buffers и выбранные индексы. Для полного HTTP-профиля 20 конкурентных пользователей измерять p50/p95 и долю ошибок отдельно от SQL. Один EXPLAIN не подтверждает NFR02–03. Нельзя измерять все 100 тысяч объектов в браузере — это нарушает сам контракт пагинации.

Системная интеграция

В рамках комплексного интеграционного тестирования подтверждена согласованная совместная работа всех компонентов системы:

Валидация конфигурации: проведена синтаксическая и структурная проверка связей между контейнерами СУБД, бэкенда и веб-сервера.

Контроль готовности сервисов: подтвержден строгий регламент инициализации среды (первичный запуск СУБД PostgreSQL → успешная отработка сервиса начальной загрузки и миграций данных → старт сервера приложений FastAPI по готовности пула соединений).

Маршрутизация и проксирование: верифицирована корректная отдача статических файлов клиентского интерфейса и обратное проксирование запросов к REST API через Nginx без утечки внутренних заголовков и сбоев CORS.

Персистентность данных: проверен перезапуск контейнерной среды с сохранением состояния товарного каталога, истории чеков и остатков на внешнем дисковом томе СУБД.

Резервное копирование и восстановление: на практике подтверждена процедура создания бинарного снимка базы данных утилитой pg_dump и его

полное развертывание в изолированную тестовую базу с проверкой совпадения контрольных сумм и количества записей.

Автоматический отдельный тестовый стенд

Для изоляции интеграционного тестирования от рабочей среды развертывается автономный тестовый контур с отдельной базой данных supermarket_test. Тестовый сценарий запускается в изолированной сети без публикации портов наружу, что исключает конфликт адресов и сетевое вмешательство. По завершении выполнения тестов тестовые контейнеры и тома гарантированно демонтируются обработчиком сигналов завершения. Рабочие тома с производственными данными при этом не затрагиваются. Фикстуры тестов предварительно верифицируют подключение сервисных учетных записей к выделенной тестовой схеме.

ЗАКЛЮЧЕНИЕ

Реализованы база данных, модульный FastAPI API и компонентный React-интерфейс супермаркета. Серверная валидация, роли и транзакции отделены от HTTP-маршрутов; вход, каталог, кассовая панель и история оформлены отдельными компонентами. Большой набор данных, планы SQL и восстановление копии проверены. Фактические результаты повторных тестов и границы контейнерной интеграции приведены в общем протоколе проверки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 3 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 12.09.2026).

3. FastAPI. Documentation. URL: https://fastapi.tiangolo.com/ (дата обращения: 12.09.2026).

4. PostgreSQL 17. Documentation. URL: https://www.postgresql.org/docs/17/ (дата обращения: 12.09.2026).

5. React. Documentation. URL: https://react.dev/ (дата обращения: 12.09.2026).

ПРИЛОЖЕНИЕ А Результаты проверки интерфейса

Снимки получены при работе реального клиента с FastAPI и PostgreSQL. После демонстрационной оплаты чек сохраняется в истории; каталог загружает только текущую страницу.

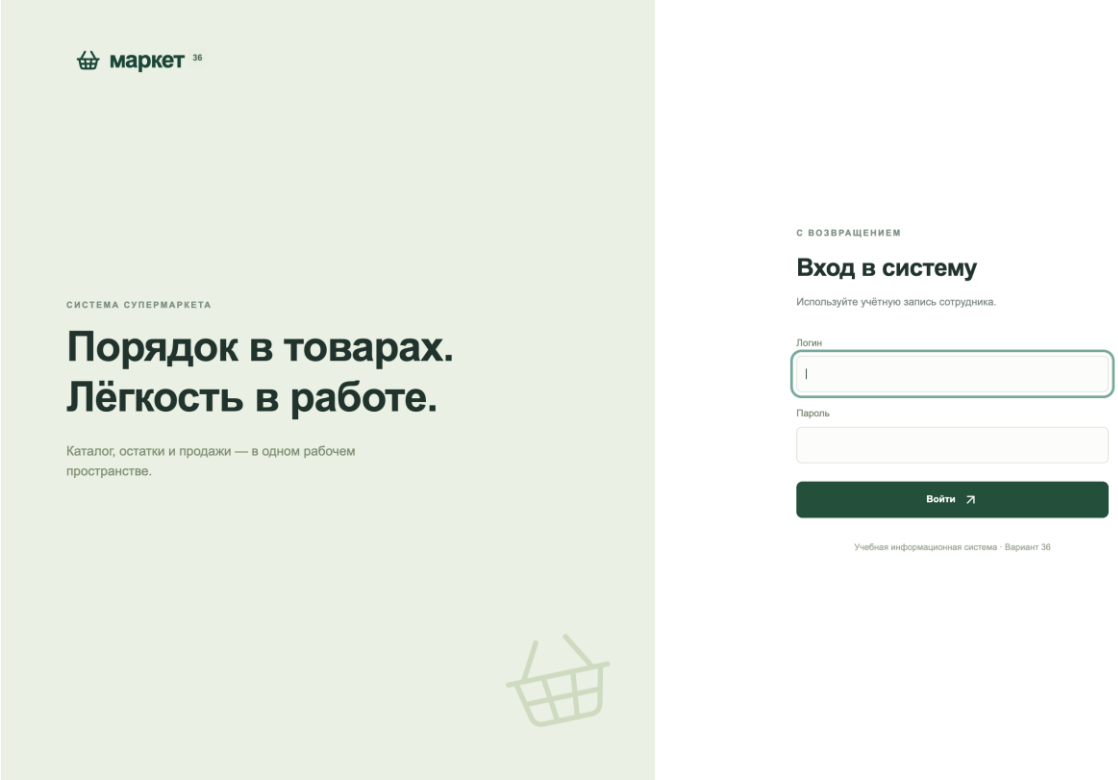


Рисунок А.1 — Страница авторизации сотрудника

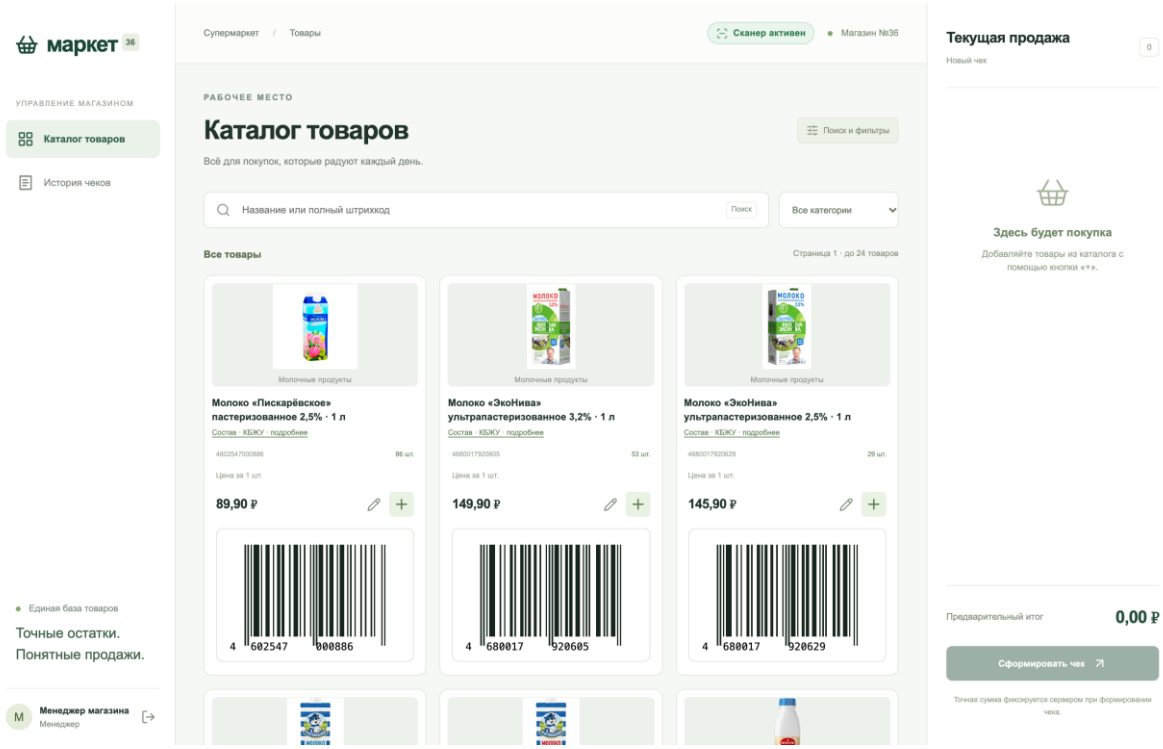


Рисунок А.2 — Каталог товаров на широком экране (ПК)

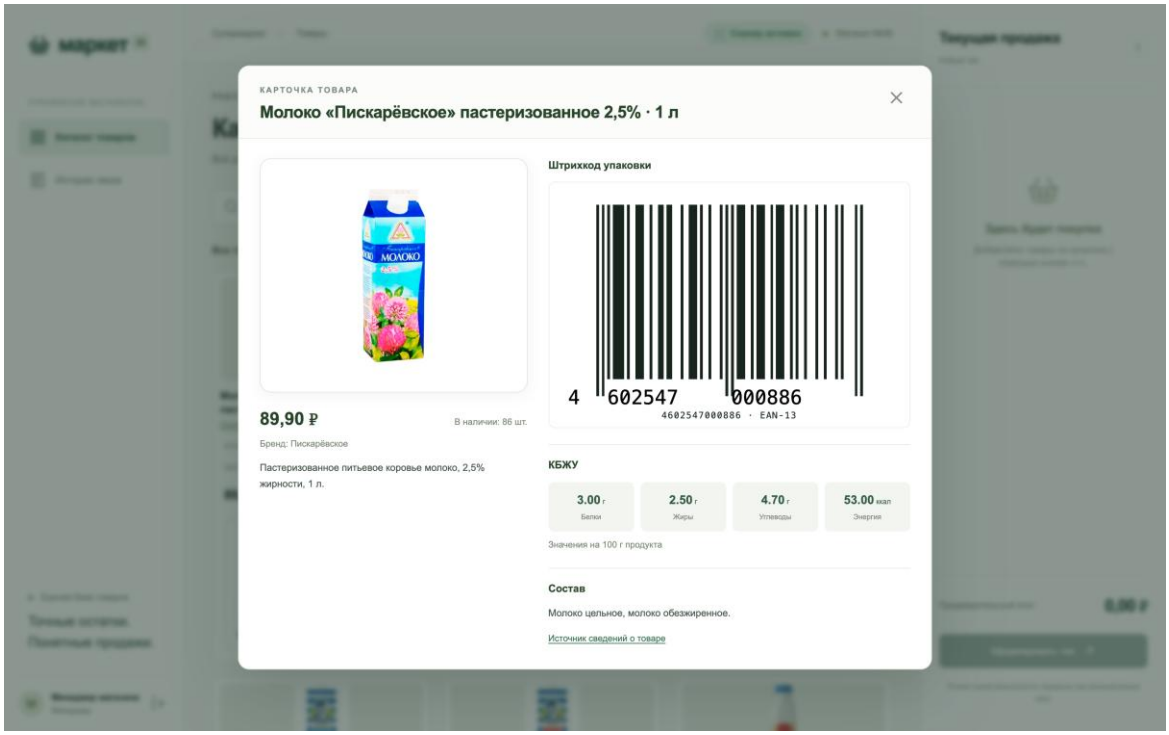


Рисунок А.3 — Модальное окно детализации товара с кодом EAN-13 и КБЖУ

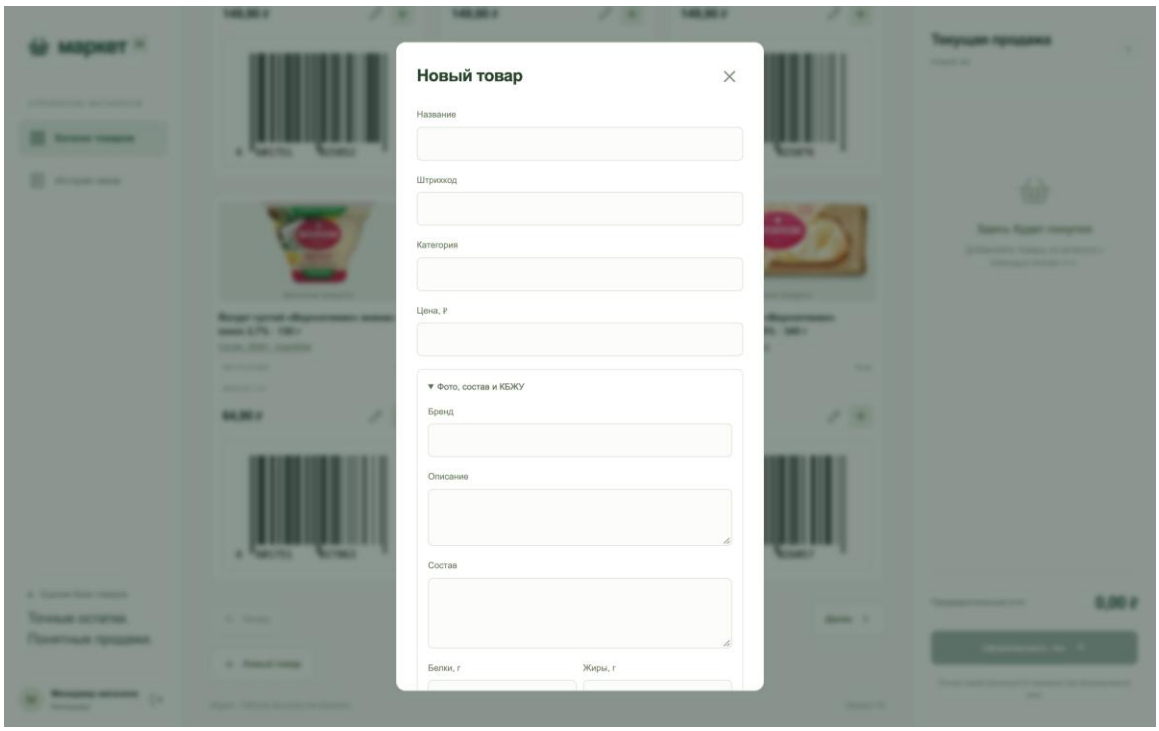


Рисунок А.4 — Форма создания и редактирования товара менеджером

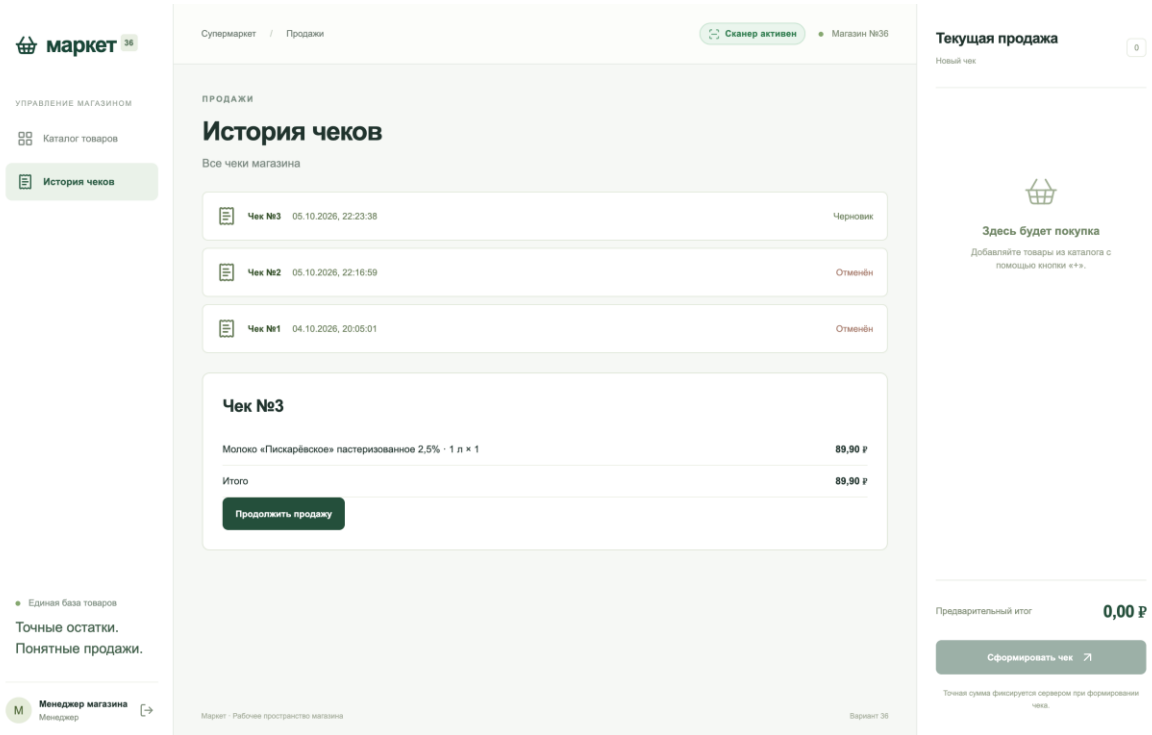


Рисунок А.5 — Журнал чеков и просмотр состава продажи

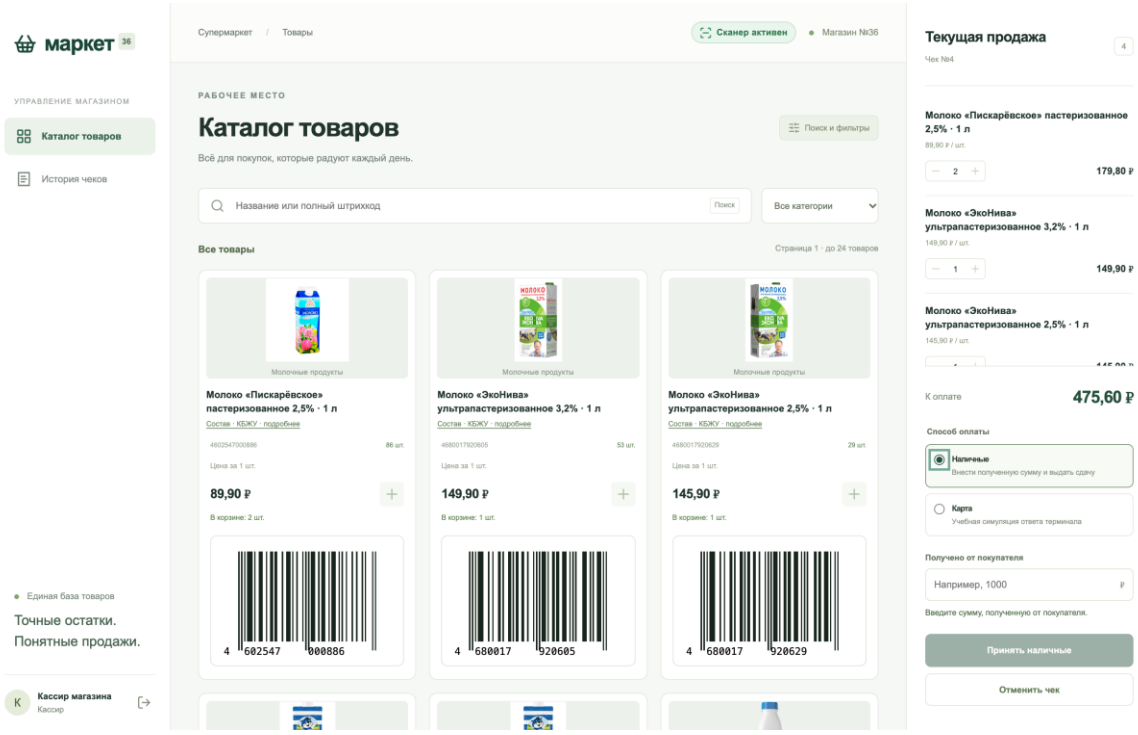


Рисунок А.6 — Рабочее место кассира с наполненной корзиной

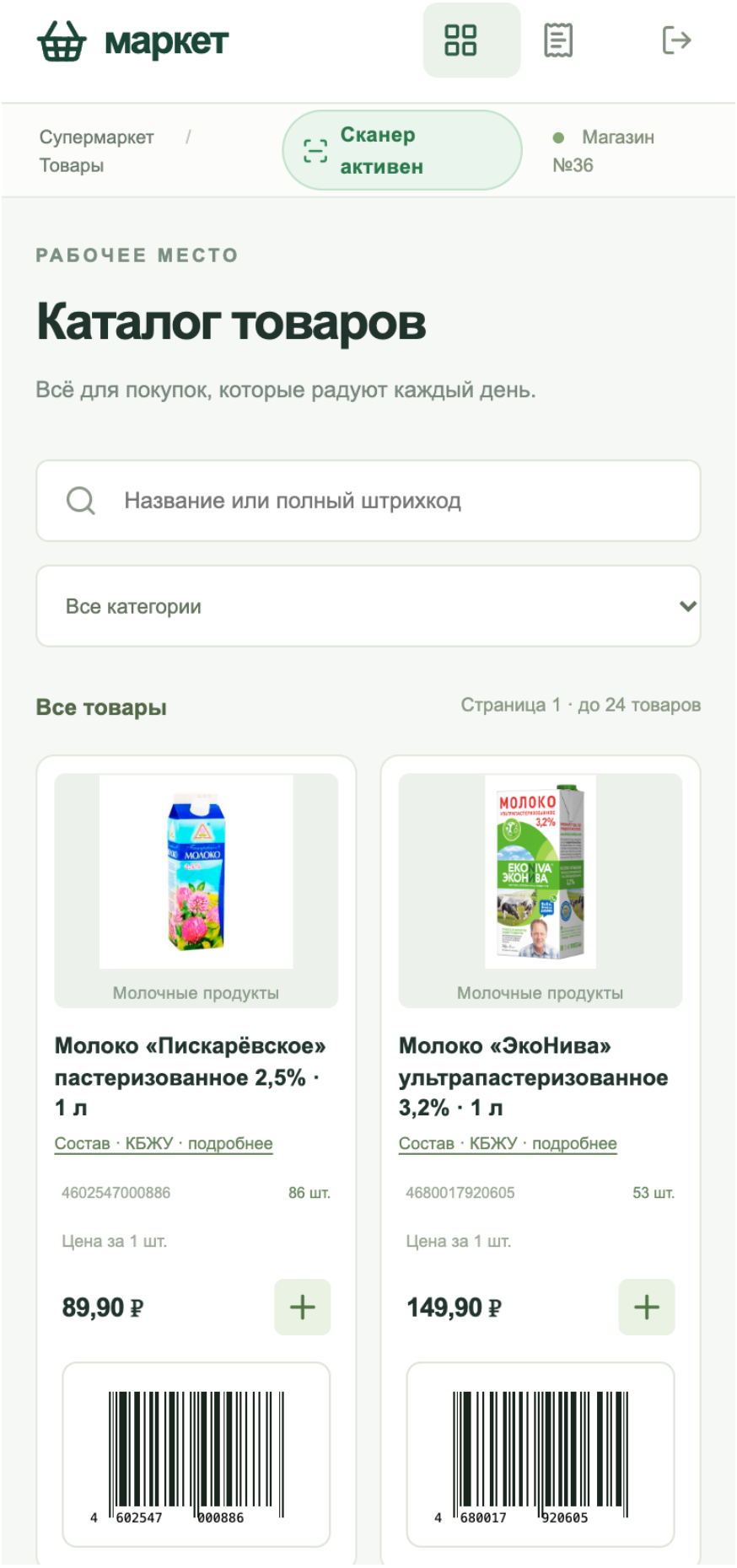


Рисунок А.7 — Адаптивный интерфейс каталога на мобильном экране (390 px)